

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

```

int main()
{
    char buffer[100];
    int n;
    printf("Enter some text: ");
    n = read(0, buffer, sizeof(buffer) - 1);
    buffer[n] = '\0'; // Null-terminate the string
    printf("You entered: %s", buffer);
    return 0;
}

```

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

#include <unistd.h>

```

int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}

```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

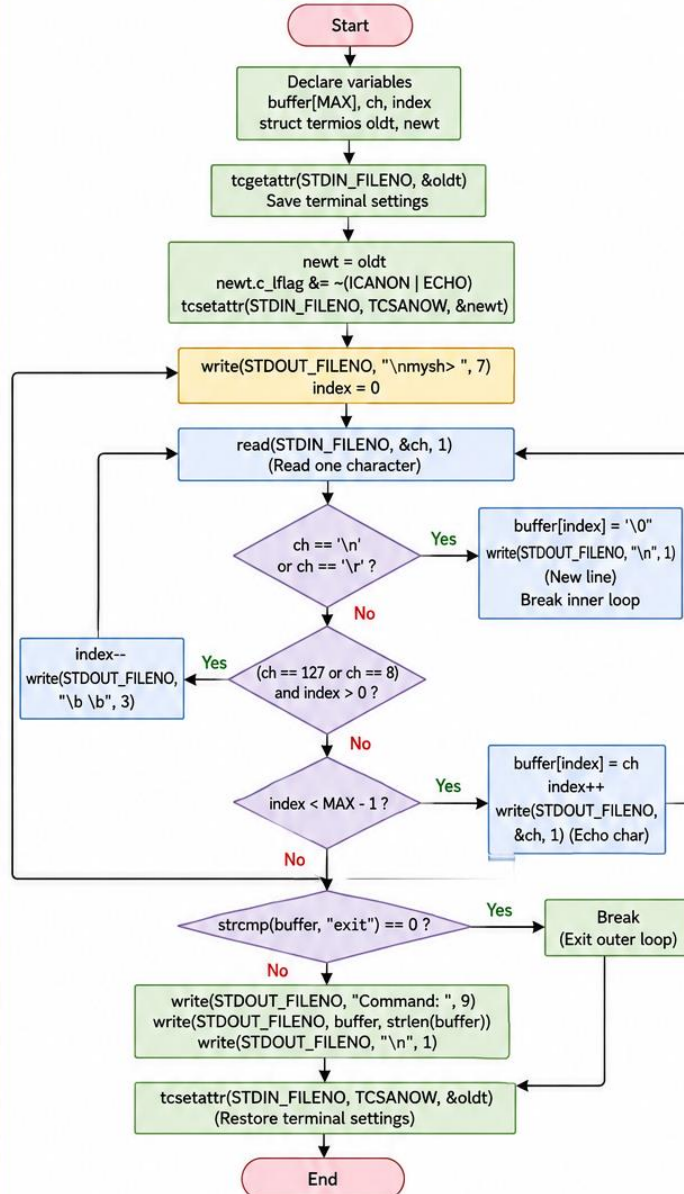
ALGORITHM & FLOWCHART

Keyboard Input Handling Using System Calls (read, write, termios)

ALGORITHM

1. Start.
2. Declare variables:
 - buffer[MAX]
 - char ch
 - int index
 - struct termios oldt, newt
3. Save the current terminal settings using `tcgetattr(STDIN_FILENO, &oldt)`.
4. Copy the terminal settings:
`newt = oldt`
5. Disable canonical mode and echo:
`newt.c_lflag &= ~(ICANON | ECHO)`
6. Apply the new terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &newt)`.
7. Repeat forever
 - (a) Display the prompt using `write()`:
"mysh> "
 - (b) Set index = 0.
 - (c) Read characters one by one using `read()`.
 - (d) If the character is Enter ('\n' or '\r'):
 - `buffer[index] = '\0'`
 - Print new line.
 - Exit the inner loop.
 - (e) Else if the character is Backspace (ASCII 8 or 127) and index > 0:
 - `index--`
 - Print "\b \b" to remove last character.
 - (f) Else if index < MAX - 1:
 - Store the character in buffer.
 - `index++`
 - Echo the character using `write()`.
 - (g) After exiting the inner loop, compare the command in buffer.
 - If command is "exit", break the outer loop.
 - Otherwise, print "Command: " followed by the command.
8. Restore the original terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &oldt)`.
9. End.

FLOWCHART



SYSTEM CALLS / FUNCTIONS USED

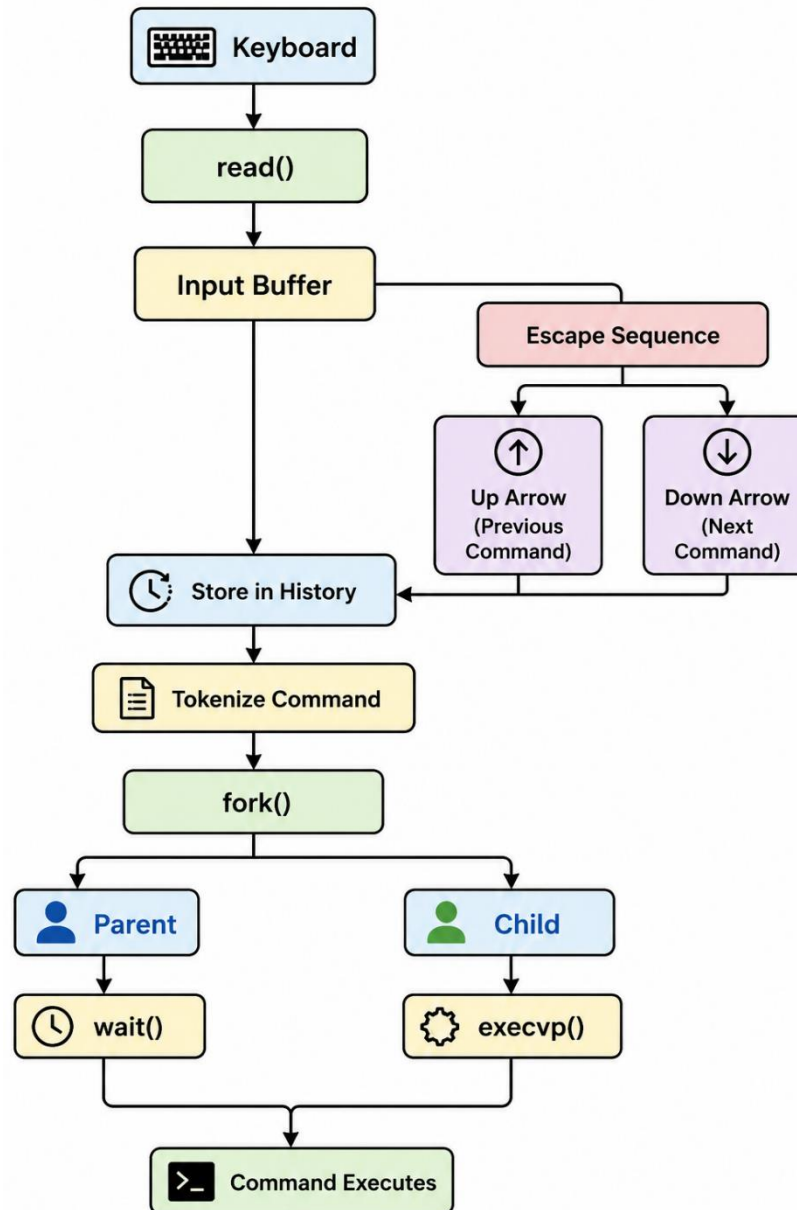
Function / System Call	Purpose
<code>read()</code>	Reads one character from keyboard (stdin).
<code>write()</code>	Writes output to terminal (stdout).
<code>tcgetattr()</code>	Gets current terminal attributes.
<code>tcsetattr()</code>	Sets terminal attributes (raw / canonical mode).
<code>strcmp()</code>	Compares strings (used to check "exit").

TERMINAL MODE

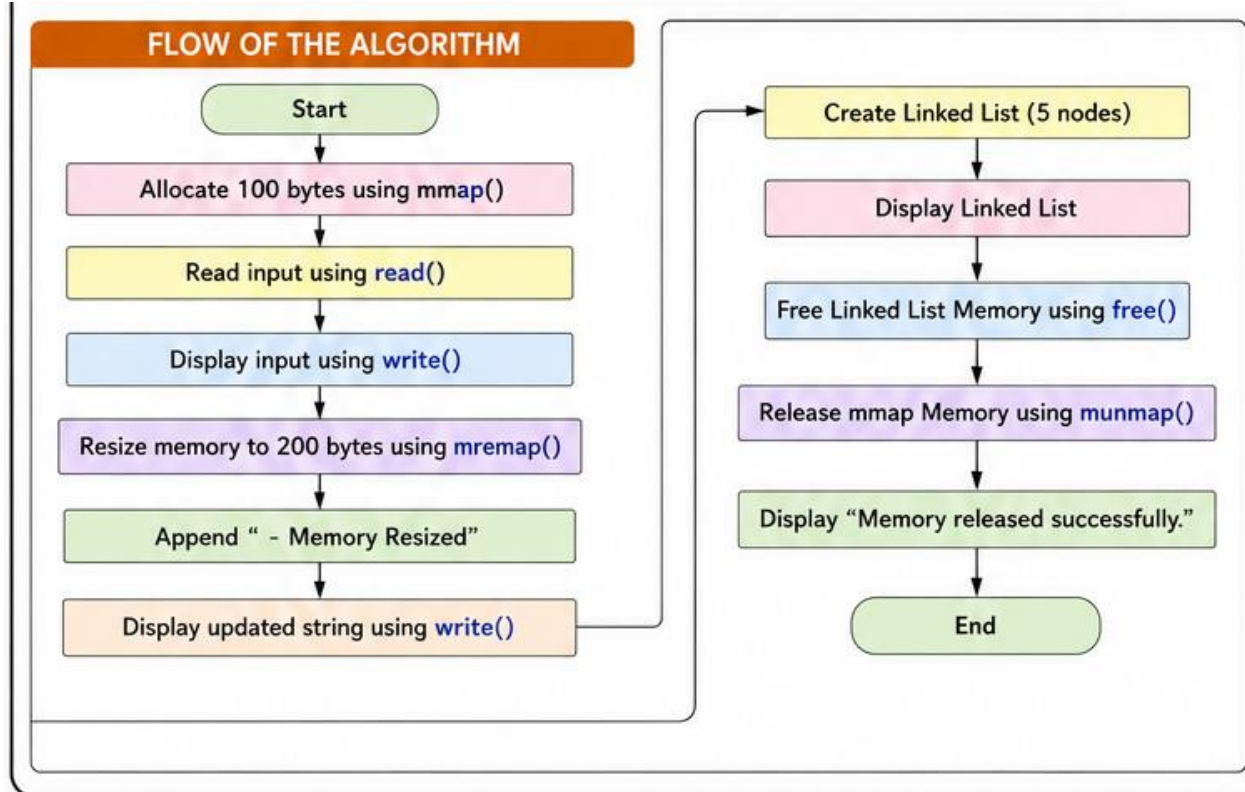
- Canonical mode (default): Input is line buffered. Backspace is handled by terminal driver.
- Non-canonical (raw) mode: Input is available character by character; program handles Backspace, Enter, etc.
- We disable ICANON (canonical) and ECHO to read characters immediately and handle them in the program.

This program uses system calls (read, write) and terminal control (termios) to capture keyboard input, handle backspace, process enter key, manage input buffer and support multi-character commands.

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality



Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.



Validate the program with valgrind tool

MY SCREENSHOTS:

Task 1 –

1. Created a new folder for week 3

```
ashborn@DESKTOP-I4G5Q4N: ~/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ pwd
/home/ashborn/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

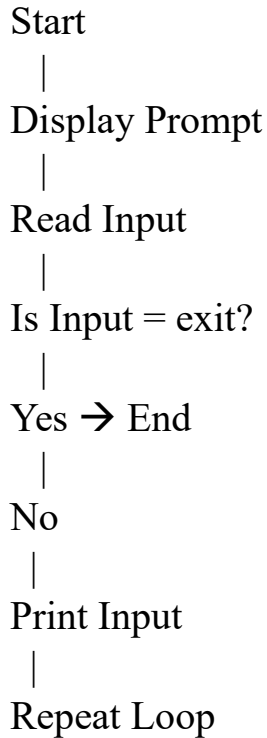
2. Created task1.c and compiled successfully

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ pwd
/home/ashborn/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task1.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task1.c -o task1
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

3. Executed task1 program

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task1.c -o task1
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task1
Enter a command (type exit to quit): hello
You entered: hello
Enter a command (type exit to quit): ubuntu
You entered: ubuntu
Enter a command (type exit to quit): ls
You entered: ls
Enter a command (type exit to quit): exit
Exiting program...
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

CONTROL FLOW DIAGRAM –



TASK 2 –

1. Created a file and shown its content

```
Exiting program...
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano File1.txt
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ cat File1.txt
Hello Ubuntu
This is Week 3 Skill Lab
Learning System Calls
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

2. Created another file and compiled successfully

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task2.c -o task2
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

3. Copied file

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task2.c -o task2
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task2
File copied successfully.
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

4. Verify

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano File1.txt
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ cat File1.txt
Hello Ubuntu
This is Week 3 Skill Lab
Learning System Calls
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task2.c -o task2
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task2
File copied successfully.
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ cat File2.txt
Hello Ubuntu
This is Week 3 Skill Lab
Learning System Calls
```


5. Final directory structure

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls -l
total 33
-rw-r--r-- 1 ashborn ashborn  60 Aug  5 06:42 File1.txt
-rw-r--r-- 1 ashborn ashborn  60 Aug  5 06:51 File2.txt
-rwxr-xr-x 1 ashborn ashborn 16184 Aug  5 06:39 task1
-rw-r--r-- 1 ashborn ashborn  390 Aug  5 06:38 task1.c
-rwxr-xr-x 1 ashborn ashborn 16176 Aug  5 06:50 task2
-rw-r--r-- 1 ashborn ashborn  525 Aug  5 06:50 task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

TASK 3 –

1. We are in our required repository

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ pwd
/home/ashborn/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

2. Created and compiled task3 successfully

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task3.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task3.c -o task3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c task3 task3.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

3. Executed task3

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task3

mysh> hello
Command: hello

mysh> ls
Command: ls

mysh> nano
Command: nano

mysh> exit
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

4. List

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls -l
total 53
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:42 File1.txt
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:51 File2.txt
-rwxr-xr-x 1 ashborn ashborn 16184 Aug 5 06:39 task1
-rw-r--r-- 1 ashborn ashborn 390 Aug 5 06:38 task1.c
-rwxr-xr-x 1 ashborn ashborn 16176 Aug 5 06:50 task2
-rw-r--r-- 1 ashborn ashborn 525 Aug 5 06:50 task2.c
-rwxr-xr-x 1 ashborn ashborn 16232 Aug 7 08:17 task3
-rw-r--r-- 1 ashborn ashborn 1259 Aug 7 08:17 task3.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

5. Combo 1

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ pwd
/home/ashborn/SKILL_Week3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task3.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task3.c -o task3
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c task3 task3.c
```

6. Combo 2

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task3

mysh> hello
Command: hello

mysh> ls
Command: ls

mysh> nano
Command: nano

mysh> exit
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls -l
total 53
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:42 File1.txt
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:51 File2.txt
-rwxr-xr-x 1 ashborn ashborn 16184 Aug 5 06:39 task1
-rw-r--r-- 1 ashborn ashborn 390 Aug 5 06:38 task1.c
-rwxr-xr-x 1 ashborn ashborn 16176 Aug 5 06:50 task2
-rw-r--r-- 1 ashborn ashborn 525 Aug 5 06:50 task2.c
-rwxr-xr-x 1 ashborn ashborn 16232 Aug 7 08:17 task3
-rw-r--r-- 1 ashborn ashborn 1259 Aug 7 08:17 task3.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

TASK 4 –

1. Created and compiled the file

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task4.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task4.c -o task4
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c task3 task3.c task4 task4.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

2. Executed the program successfully

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task4
Simple Command History
Type 'exit' to quit.

Command> hello
Stored: hello
History:
1. hello

Command> ubuntu
Stored: ubuntu
History:
1. hello
2. ubuntu

Command> linux
Stored: linux
History:
1. hello
2. ubuntu
3. linux

Command> exit
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```

3. Final directory

```

ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls -l
total 73
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:42 File1.txt
-rw-r--r-- 1 ashborn ashborn 60 Aug 5 06:51 File2.txt
-rwxr-xr-x 1 ashborn ashborn 16184 Aug 5 06:39 task1
-rw-r--r-- 1 ashborn ashborn 390 Aug 5 06:38 task1.c
-rwxr-xr-x 1 ashborn ashborn 16176 Aug 5 06:50 task2
-rw-r--r-- 1 ashborn ashborn 525 Aug 5 06:50 task2.c
-rwxr-xr-x 1 ashborn ashborn 16232 Aug 7 08:17 task3
-rw-r--r-- 1 ashborn ashborn 1259 Aug 7 08:17 task3.c
-rwxr-xr-x 1 ashborn ashborn 16272 Aug 7 08:35 task4
-rw-r--r-- 1 ashborn ashborn 861 Aug 7 08:35 task4.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$

```

TASK 5 –

1. Created and compiled the file task5 successfully

```

ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ nano task5.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc -task5.c -o task5
gcc: error: unrecognized command-line option '-task5.c'
gcc: fatal error: no input files
compilation terminated.
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ gcc task5.c -o task5
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c

```

2. Executed the program

```

ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ./task5
Enter text: Hello Ubuntu
You entered: Hello Ubuntu
Hello Ubuntu - Memory Resized
Memory released successfully.

```

3. Installed valgrind successfully

```
ashborn@DESKTOP-I4G5Q4N:~$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libdw
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind alleyoop valkyrie
The following NEW packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libip
The following packages will be upgraded:
  libdw1t64 libelf1t64
2 upgraded, 9 newly installed, 0 to remove and 222 not upgraded.
Need to get 27.3 MB of archives.
After this operation, 103 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginf
```

4. Executed the program using valgrind

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ valgrind ./task5
==454== Memcheck, a memory error detector
==454== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==454== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==454== Command: ./task5
==454==
==454== error calling PR_SET_PTRACER, vgdb might block
Enter text: Hello Ubuntu
You entered: Hello Ubuntu
Hello Ubuntu - Memory Resized
Memory released successfully.
==454==
==454== HEAP SUMMARY:
==454==      in use at exit: 0 bytes in 0 blocks
==454==    total heap usage: 4 allocs, 4 frees, 1,324 bytes allocated
==454==
==454== All heap blocks were freed -- no leaks are possible
==454==
==454== For lists of detected and suppressed errors, rerun with: -s
==454== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

5. Final directory

```
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$ ls -l
total 93
-rw-r--r-- 1 ashborn ashborn   60 Aug  5 06:42 File1.txt
-rw-r--r-- 1 ashborn ashborn   60 Aug  5 06:51 File2.txt
-rwxr-xr-x 1 ashborn ashborn 16184 Aug  5 06:39 task1
-rw-r--r-- 1 ashborn ashborn   390 Aug  5 06:38 task1.c
-rwxr-xr-x 1 ashborn ashborn 16176 Aug  5 06:50 task2
-rw-r--r-- 1 ashborn ashborn   525 Aug  5 06:50 task2.c
-rwxr-xr-x 1 ashborn ashborn 16232 Aug  7 08:17 task3
-rw-r--r-- 1 ashborn ashborn   1259 Aug  7 08:17 task3.c
-rwxr-xr-x 1 ashborn ashborn 16272 Aug  7 08:35 task4
-rw-r--r-- 1 ashborn ashborn    861 Aug  7 08:35 task4.c
-rwxr-xr-x 1 ashborn ashborn 16304 Aug  8 08:30 task5
-rw-r--r-- 1 ashborn ashborn    666 Aug  8 08:30 task5.c
ashborn@DESKTOP-I4G5Q4N:~/SKILL_Week3$
```